Parallel Monte Carlo

Four implementations. One laptop.
Two questions: how fast, and how reliable?

Rajrup Mal — MPI
Dev Visoulia — OpenMP
Aryan Sheikh — CUDA & integration

CSE (AI & ML) · AIML-C2

Estimate an area by counting points

π estimate = 4 × inside points / total points

Generate independent uniform points (x, y).

Count points where $x^2 + y^2 \le 1$.

Typical sampling error decreases as $1 / \sqrt{N}$.

Motivation: numerical integration, uncertainty analysis and particle simulation.

What this project contributes

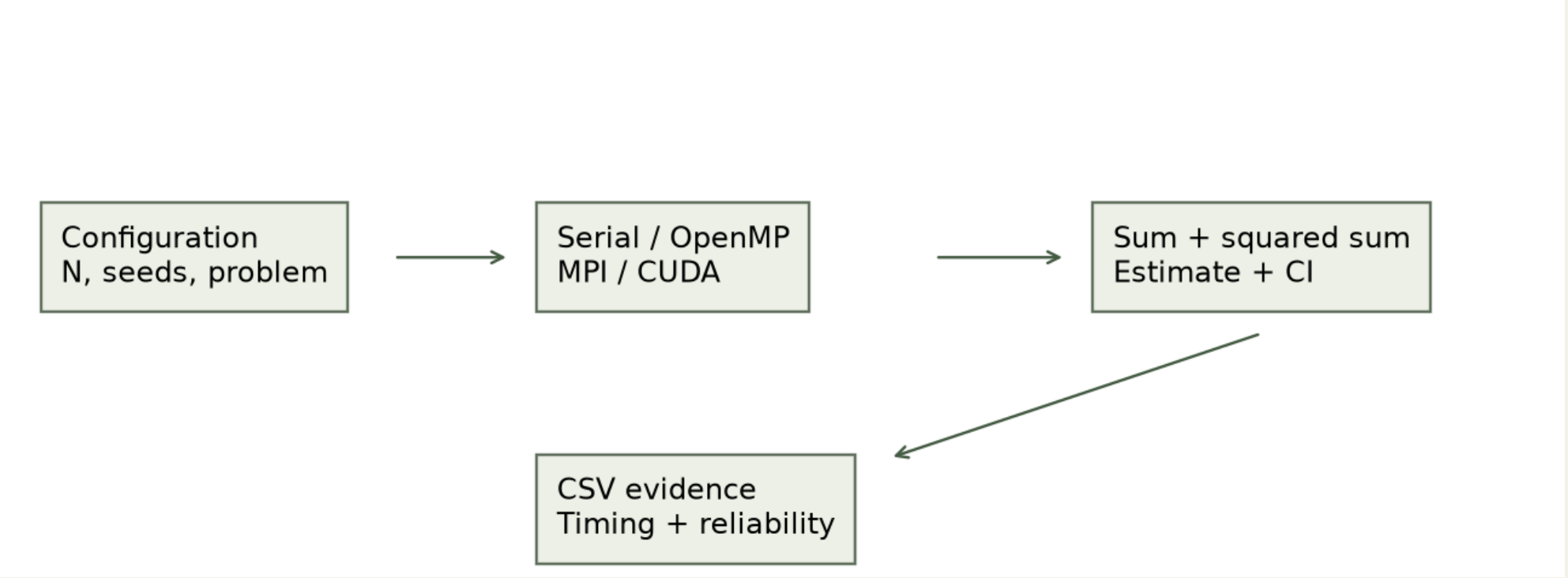
Compare sequential C++, OpenMP, MPI and CUDA.

Separate execution speed from estimate quality.

Test separated streams against deliberate stream reuse.

Connect timing to GPU counters and GPU-only energy.

One experiment, four execution paths



The same statistical output makes correctness checks and comparisons possible.

How work is divided

Sequential: one CPU execution path processes every chunk.

OpenMP: threads share memory and reduce partial sums.

MPI: local processes own chunks and combine results.

CUDA: GPU threads sample points and reduce block totals.

MPI was tested on one laptop; these are not multi-node cluster results.

A fast estimate can still be unreliable

CPU: MT19937-64 streams assigned to fixed chunks.

GPU: Philox subsequences assigned to global thread IDs.

Negative control: reuse streams across chunks or blocks.

Repeated samples contain less information than fresh samples.

CPU and GPU use different RNG families: their speedup is an implementation comparison.

Measure under stated conditions

Intel Core Ultra 7 255HX + RTX 5060 Laptop GPU.

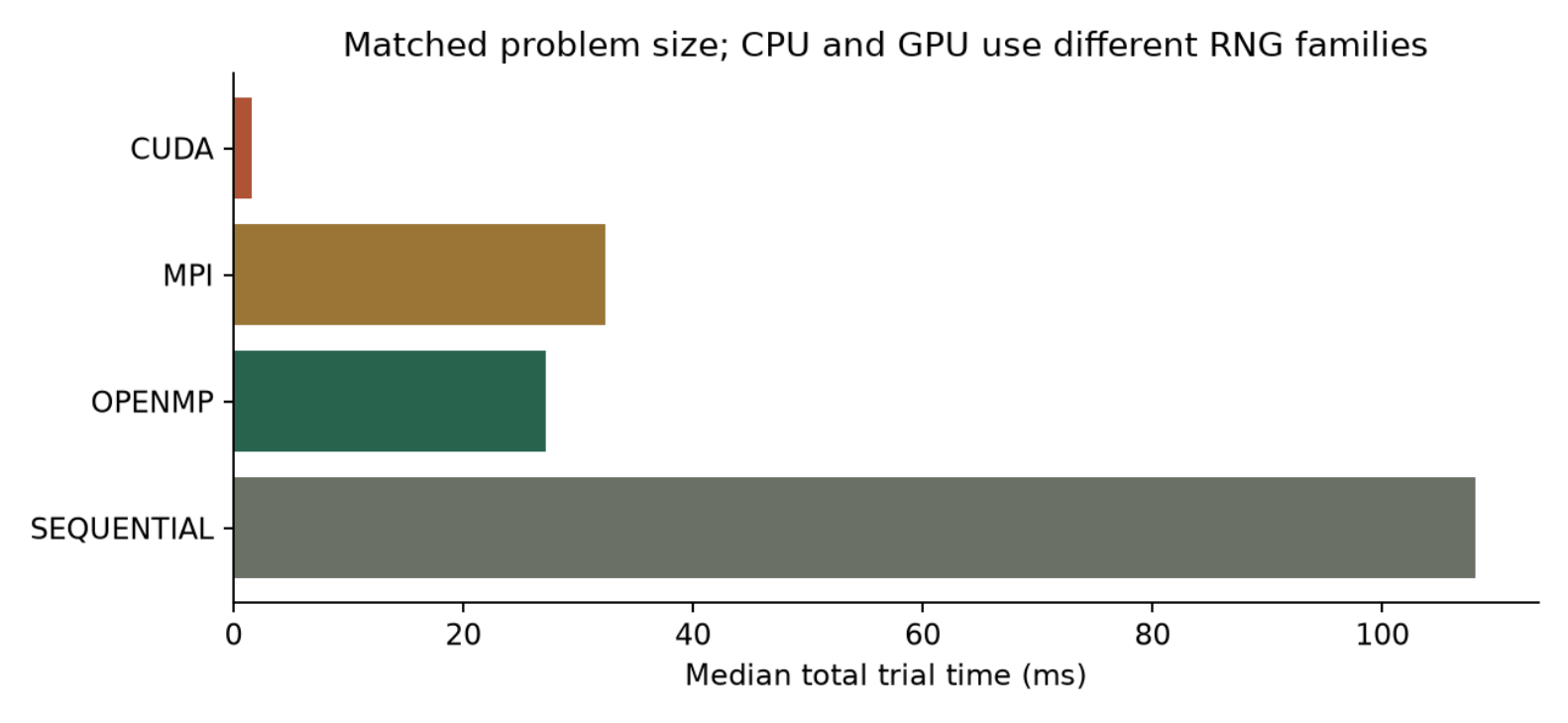
Common comparison: 30 trials per configuration.

Three shuffled rounds reduce fixed-order timing effects.

Report total trial time separately from GPU kernel time.

MPI startup and process launch are outside the reported trial timing.

Ten million samples: measured comparison



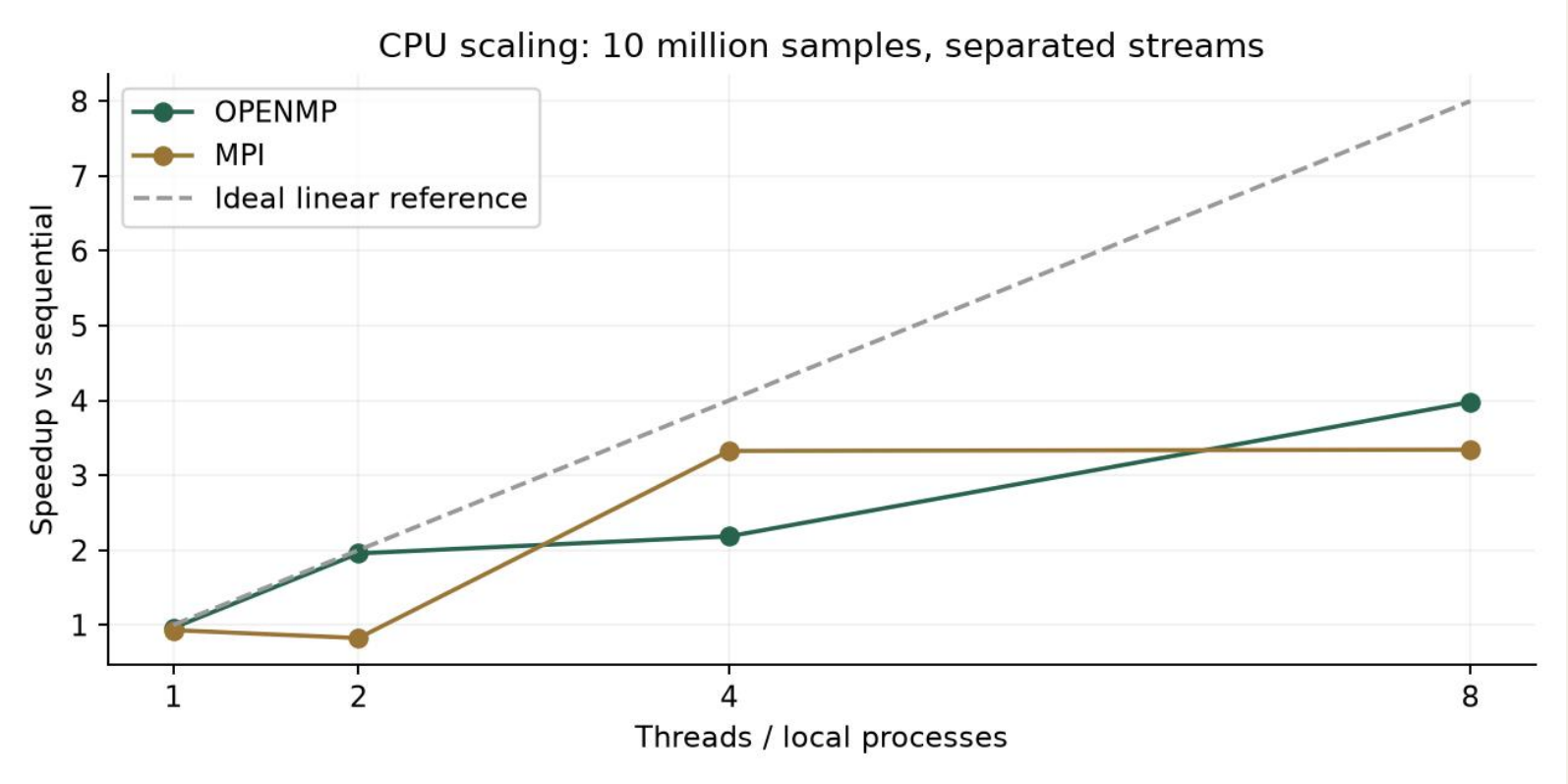
3.98× OpenMP

3.34× MPI

68.00× CUDA

Eight CPU workers / ranks · CUDA block 256 · total trial timing

Scaling is useful—and imperfect



8-worker efficiency

OpenMP 49.7%
MPI 41.8%

Do not use CUDA thread count as an efficiency denominator.

The two-rank MPI slowdown remains a limitation to investigate.

Reliability: test intervals across seeds

Separated streams

98 / 100

intervals covered π

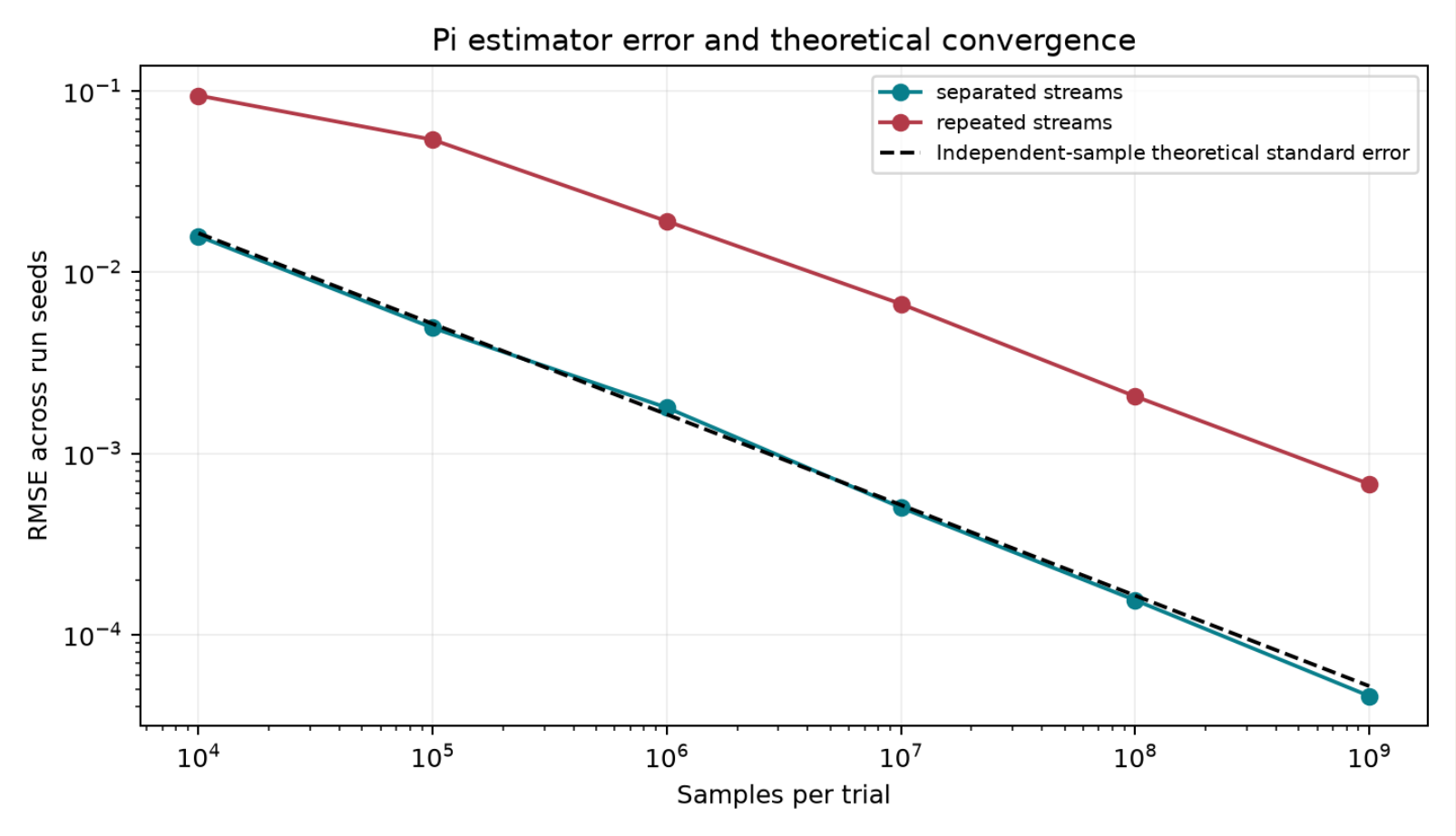
Repeated streams

12 / 100

intervals covered π

Separate CUDA study · N = 10^9 · block 256 · 100 distinct run seeds

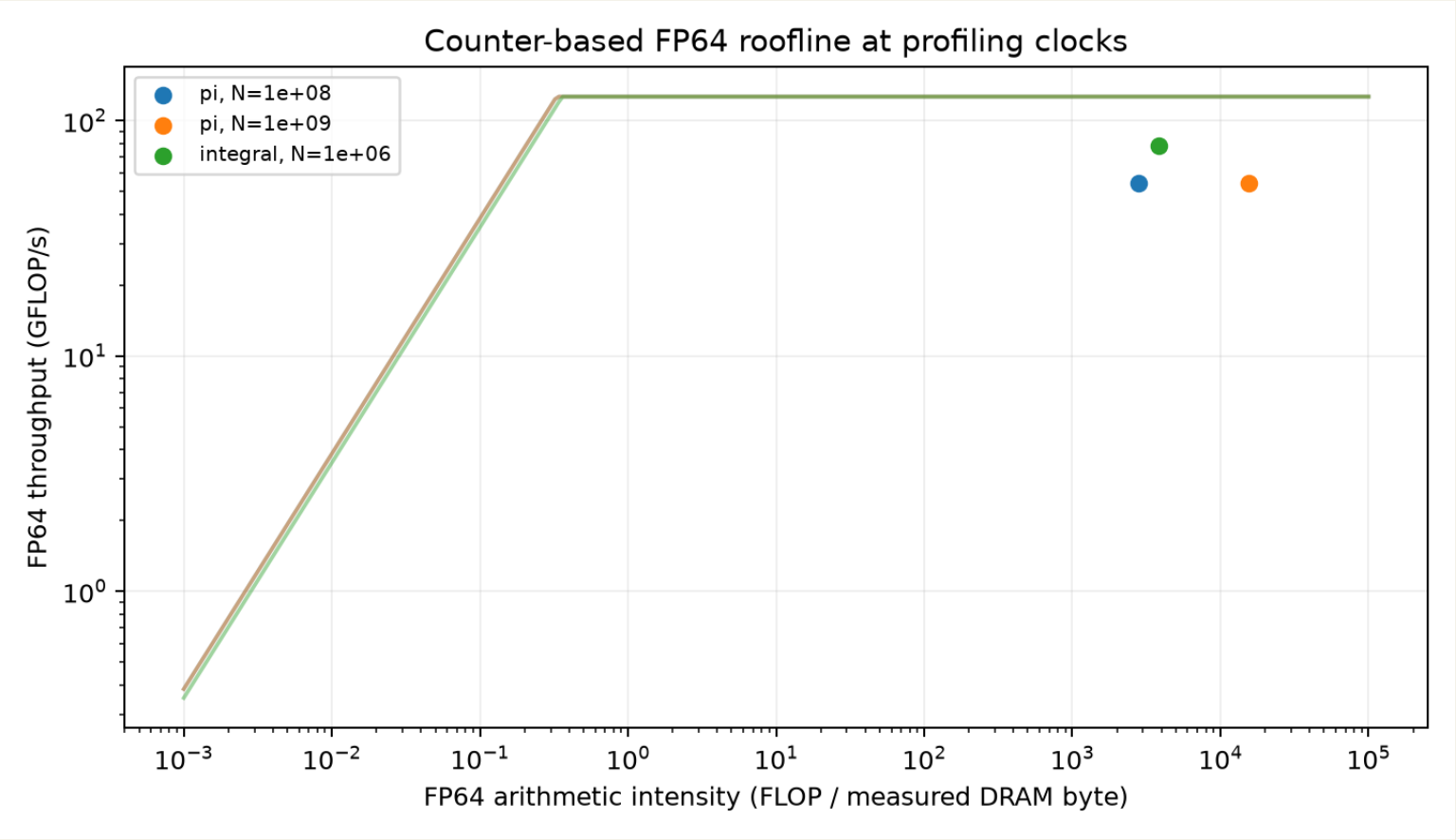
Error decreases with sample count



Check the error bars, not just the digits.

Also tested a known-answer integral in 2, 8, 16 and 32 dimensions.

What limits the tested CUDA kernels?



≈54 GFLOP/s
FP64 π kernel

≈303 million
samples / GPU joule

Counter diagnosis is configuration-specific; GPU-only energy includes the aligned workload window.

How this fits existing research

OpenMC: production transport across GPU vendors [1].

Transport RNG work: reproducible stream assignment [2].

cuVegas: adaptive multidimensional integration [3].

Our scope: a small, explainable benchmark with reliability checks.

[1] arXiv:2403.12345 [2] arXiv:2403.06362 [3] arXiv:2408.09229

Live demonstration

Open http://127.0.0.1:8765

Choose 1,000,000 samples, 5 trials and 4 workers.

Run all four backends; explain timing and estimate columns.

Hover labels for definitions, then export the measured CSV.

Keep a labelled recorded run as backup. Short demonstrations do not validate RNG quality.

Conclusions and limits

All four implementations are complete and validated.

CUDA accelerates the tested workload; RNG strategy still matters.

One laptop, different RNG families and local MPI limit generalization.

Next: matched RNGs, repeated profiles and broader hardware tests.

Backup: formulas for the viva

π estimate = $4H / N$

Theoretical variance = $(4\pi - \pi^2) / N$

Approximate 95% interval = estimate ± 1.96 × standard error

Integral: mean of d squared coordinates; exact value = 1/3

Independent samples are required for these standard-error formulas.

Backup: timing and correctness

Speedup = sequential median / parallel median.

CPU efficiency = speedup / workers.

Kernel time is only one part of CUDA total trial time.

Correctness checks cover chunk boundaries, seeds and reductions.